CS3.301: Operating Systems and Networks

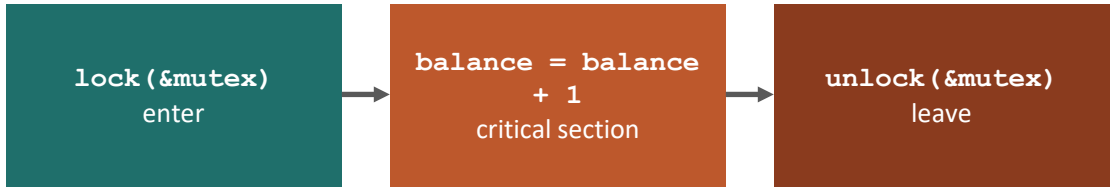
Lecture 19 – Locks

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Locks: The Basic Idea

- Goal: run a series of instructions **atomically**, even with interrupts and multiple CPUs
- Canonical critical section: **`balance = balance + 1;`**
- Programmers annotate the source, wrapping the critical section in a lock
- The lock makes the whole region behave as if it were one atomic instruction
- A lock is just a variable: declare it, then use it

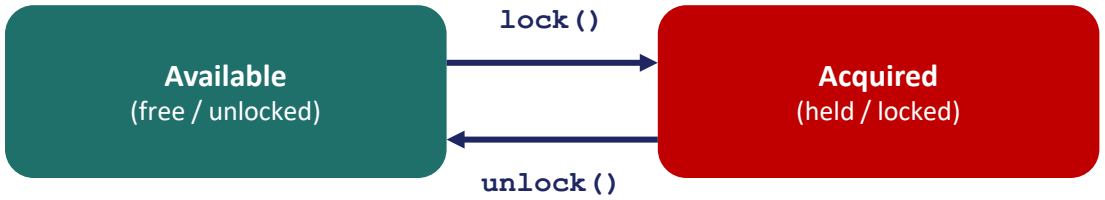
```
lock_t mutex; // some globally-allocated lock 'mutex'
...
lock(&mutex);
balance = balance + 1;
unlock(&mutex);
```

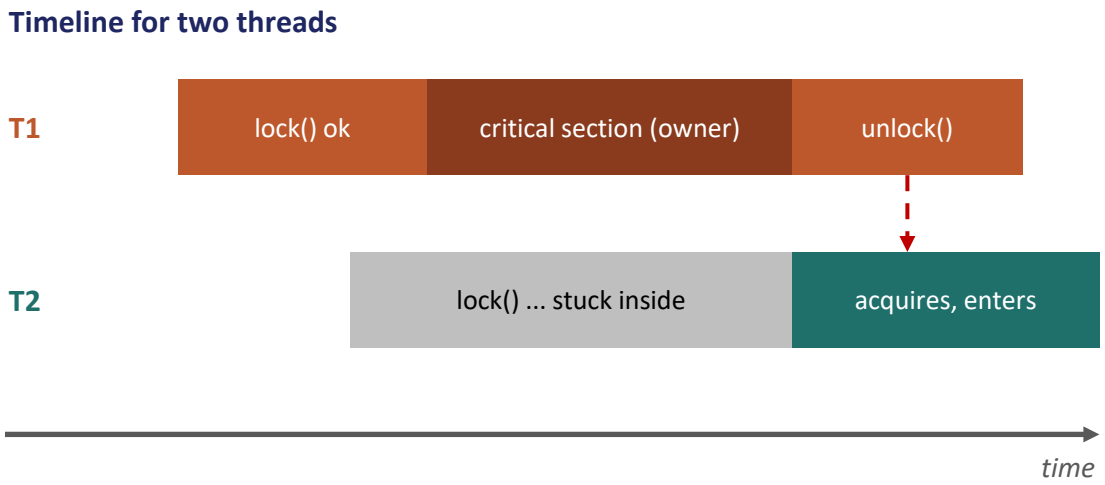


Everything between lock and unlock runs as if it were a single atomic instruction

Lock States and Semantics

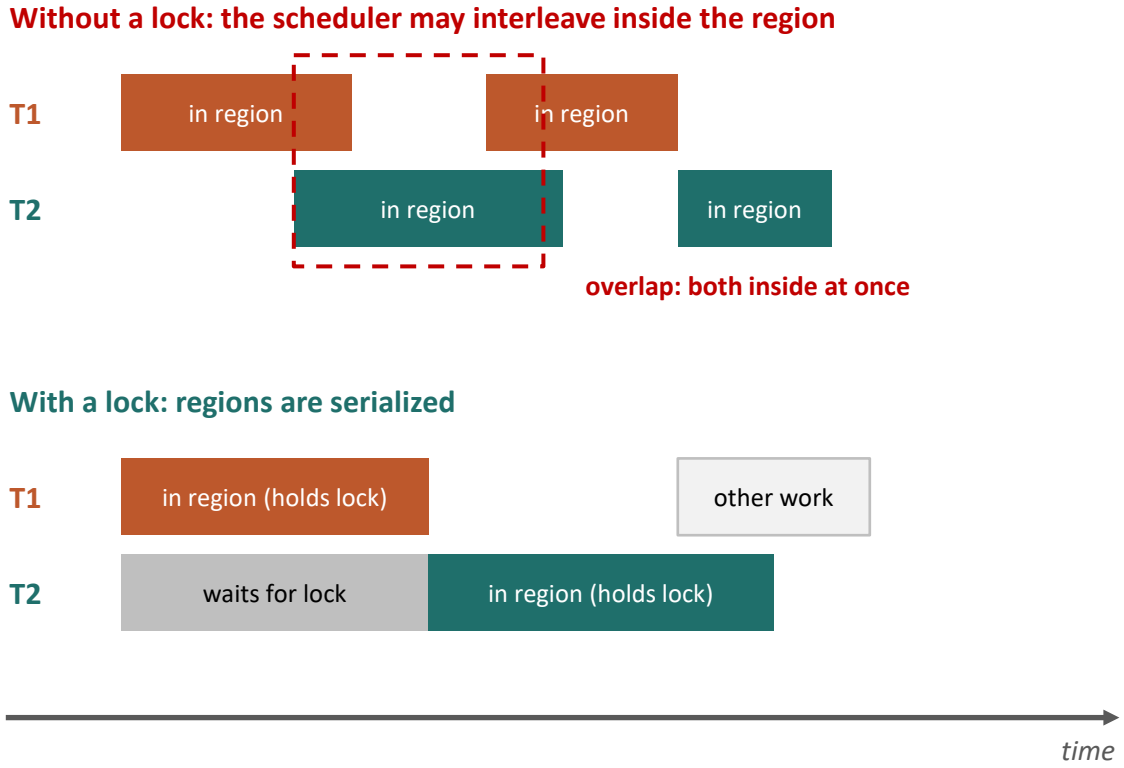
- A lock variable holds the lock's state: **available** (unlocked, free) or **acquired** (locked, held)
- Exactly one thread holds a held lock: the owner, which is inside the critical section
- lock(): if free, acquire it and enter; if held, **do not return** until the lock is free
- unlock(): lock becomes free; one waiting thread eventually notices, acquires it, and enters





Locks Give Scheduling Control Back to the Programmer

- Threads are created by the programmer but scheduled by the OS, in any fashion the OS chooses
- Locks return some of that control to the programmer
- Guarantee: no more than one thread is ever active within a locked region
- Locks turn the chaos of ordinary OS scheduling into a more controlled activity

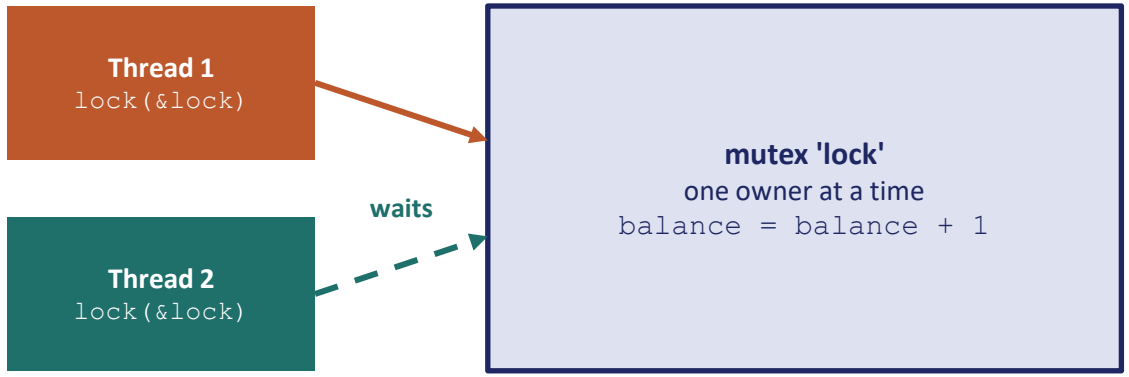


Pthread Locks: The Mutex

- POSIX calls the lock a **mutex**: it provides **mutual exclusion** between threads
- If one thread is in the critical section, it excludes the others until it completes
- Same pattern as before; we use wrappers that check for errors and **exit on failure**
- POSIX lock and unlock take a variable, so different locks can protect different data

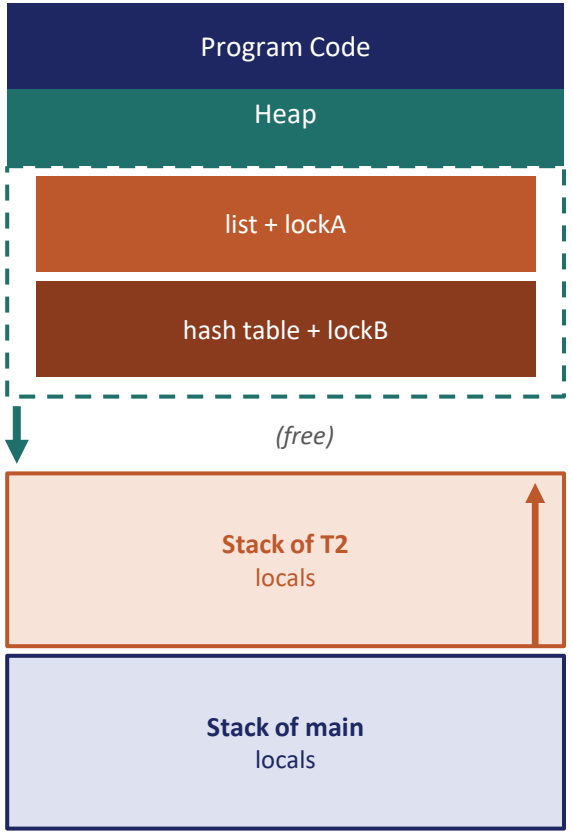
```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

Pthread_mutex_lock(&lock); // wrapper; exits on failure
balance = balance + 1;
Pthread_mutex_unlock(&lock);
```



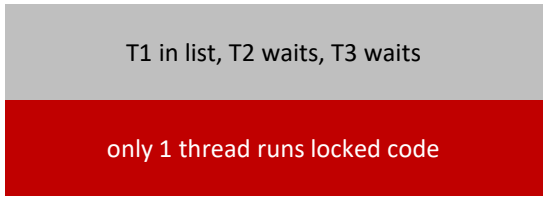
Coarse-Grained vs Fine-Grained Locking

- Different locks for different data can **increase concurrency**
- **Coarse-grained**: one big lock used whenever any critical section is accessed
- **Fine-grained**: different locks protect different data structures
- More threads can then be in locked code at once, as long as they use different locks
- Locks live in shared memory (globals or heap), so every thread can reach them



heap grows down, stacks grow up

Coarse: one lock

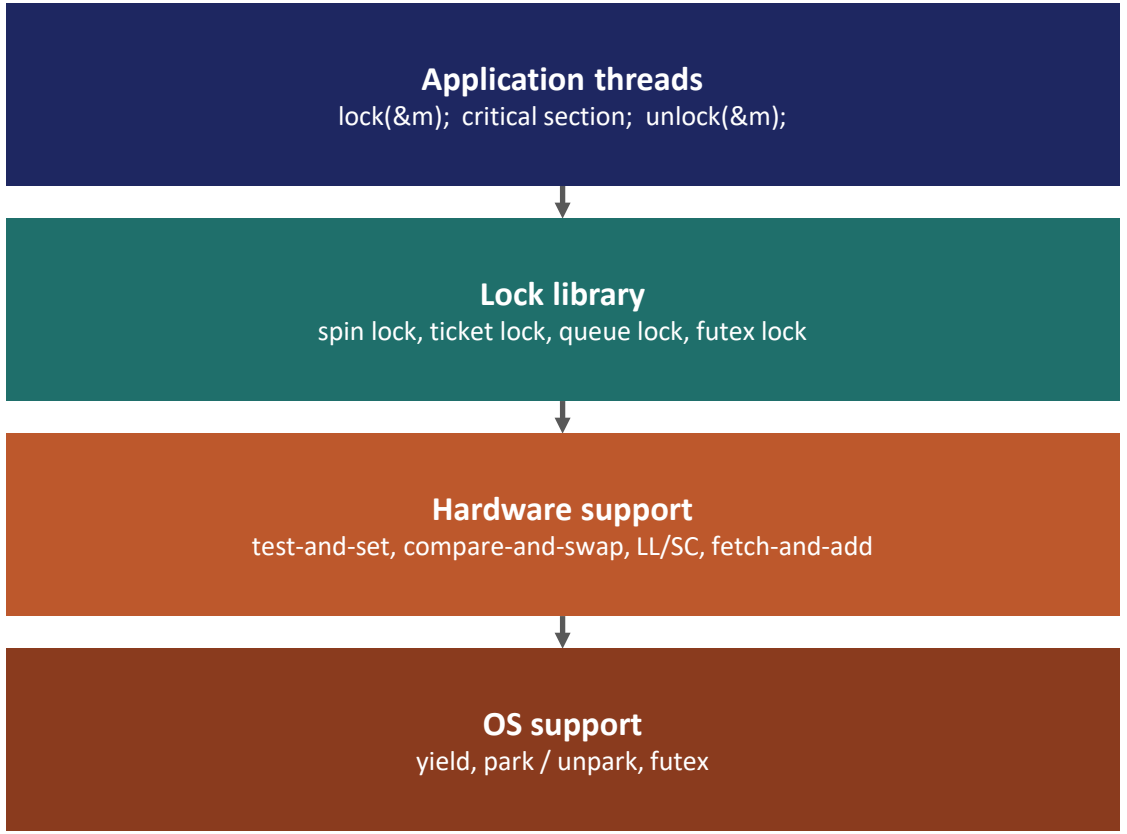


Fine: lockA and lockB



The Crux: How to Build a Lock

- We know how a lock works for the programmer; but how do we build one?
- Crux: how can we build an efficient lock?
- Efficient locks give mutual exclusion at low cost, and ideally some other good properties
- Needs help from the hardware (special instructions) and from the OS
- We study both: first hardware primitives, then OS support



Evaluating Locks: Three Criteria

1. Mutual exclusion

Does the lock do its basic job: prevent multiple threads from entering a critical section at the same time?

2. Fairness

Does each thread contending for the lock get a fair shot at acquiring it once it is free? Or can a thread starve and never obtain it?

3. Performance

What are the time overheads added by using the lock? We consider several scenarios.

A lock is only useful if it is correct first; fairness and performance decide how good it is in practice

Lock Performance: Three Scenarios

- Performance is not one number; it depends on **contention** and the number of CPUs
- Compare the techniques under the same three scenarios
- Each scenario highlights a different cost of the lock

Scenario	Question to ask
No contention	One thread runs, grabs and releases the lock: what is the overhead?
Contention, single CPU	Many threads compete on one CPU: are there performance concerns?
Contention, multiple CPUs	Threads on each CPU compete for the lock: how does the lock perform?

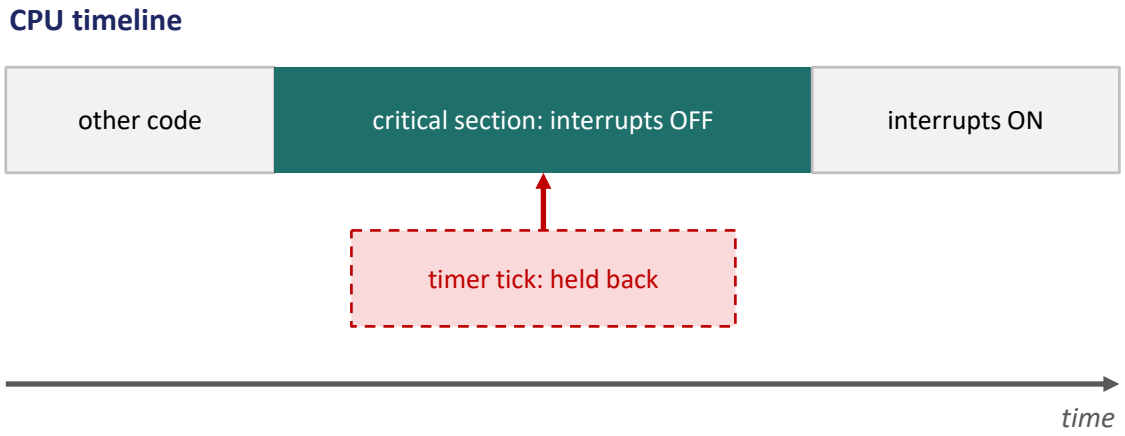


1 thread **N threads, 1 CPU** **N threads, N CPUs**

Controlling Interrupts

- One of the earliest solutions: disable interrupts around a critical section
- Invented for single-processor systems
- No interrupt means no timer tick, so no context switch: code runs as if atomic
- Main positive: **simplicity**; easy to see why it works

```
void lock() {
    DisableInterrupts();
}
void unlock() {
    EnableInterrupts();
}
```



Why Disabling Interrupts Is Not Enough

1. Requires trust

Any thread may now perform a privileged operation. A greedy program could call lock() at the start and monopolize the CPU; an errant one could loop forever and the OS could only reboot.

2. No multiprocessor support

Threads on other CPUs keep running. Disabling interrupts on one CPU does not stop another CPU from entering the same critical section.

3. Lost interrupts

Long periods with interrupts off can lose them. If the CPU misses a disk read completion, how will the OS know to wake the waiting process?

Today: used only inside the OS, where trust is not an issue, e.g. to guarantee atomicity in its own data structures

A Failed Attempt: Just Using Loads and Stores

- Rely on a single variable, flag, accessed with ordinary loads and stores
- flag = 0: lock available; flag = 1: lock held
- lock(): **test** the flag; spin while it is 1; then **set** it to 1
- unlock(): clear the flag
- Looks plausible, but it has two problems: one of correctness, one of performance

```
typedef struct __lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1) // TEST the flag
        ; // spin-wait (do nothing)
    mutex->flag = 1;         // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```

Problem 1: No Mutual Exclusion

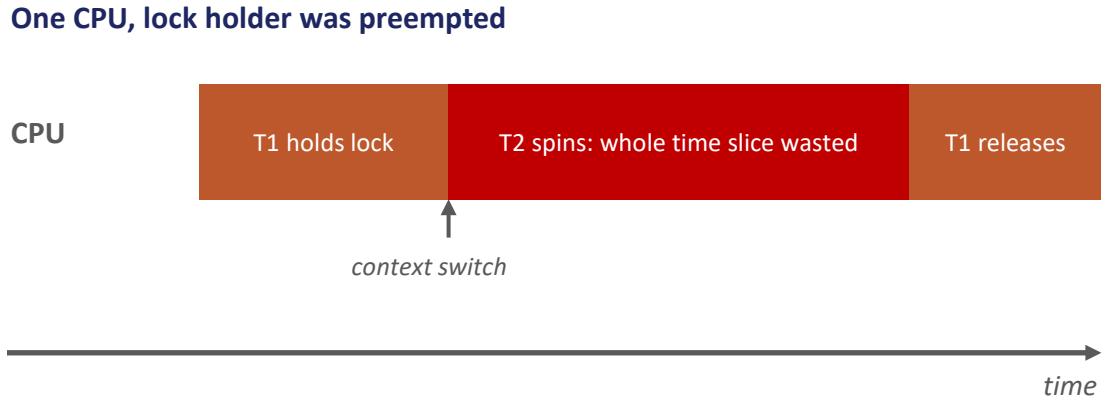
- Test and set are two separate steps, so an interrupt can arrive in between
- Assume flag = 0 to begin
- Both threads see flag == 0 and both set flag to 1
- Both enter the critical section: the **most basic requirement fails**

Thread 1	Thread 2	flag
call lock()		0
while (flag == 1) // false		0
interrupt: switch to T2		**0**
	call lock()	0
	while (flag == 1) // false	0
	flag = 1;	1
	interrupt: switch to T1	**1**
flag = 1; // too!		**1**

Both threads now believe they hold the lock

Problem 2: Spin-Waiting Wastes Time

- The waiter endlessly checks flag: this is **spin-waiting**
- On a uniprocessor the waste is exceptionally high: the thread it waits for cannot even run until a context switch
- Tip: think about concurrency as a **malicious scheduler**
- Pretend to be a scheduler that interrupts at the most inopportune moments
- If one such interleaving breaks the lock, the approach is wrong



Malicious scheduler checklist

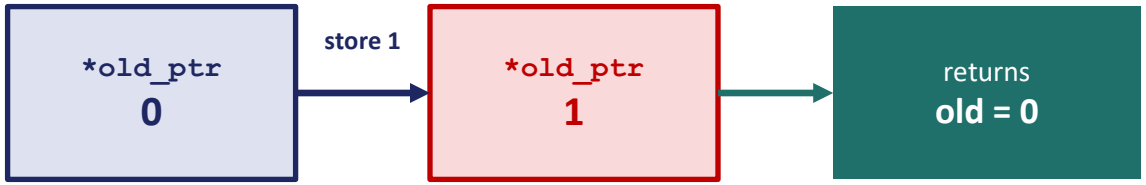
1. Where can I interrupt this code?
2. Does the other thread then break the rule?
3. Improbable is not impossible: one bad schedule is enough

Hardware Support: Test-and-Set

- Hardware designers added lock support early: early 1960s
- Simplest primitive: **test-and-set** (atomic exchange)
- Returns the old value at old_ptr and stores new, as one atomic step

```
int TestAndSet(int *old_ptr, int new) {
    int old = *old_ptr; // fetch old value at old_ptr
    *old_ptr = new;     // store 'new' into old_ptr
    return old;         // return the old value
}
```

All of this happens as one indivisible step



Nobody can sneak in between the read of the old value and the write of the new one

A Simple Spin Lock Using Test-and-Set

- Same structure as the failed flag lock
- The test and the set are now one atomic **`TestAndSet(&lock->flag, 1)`**
- Returns 1: lock was held, so keep spinning
- Returns 0: lock was free and is now atomically marked held, so we own it
- unlock: store 0 to release

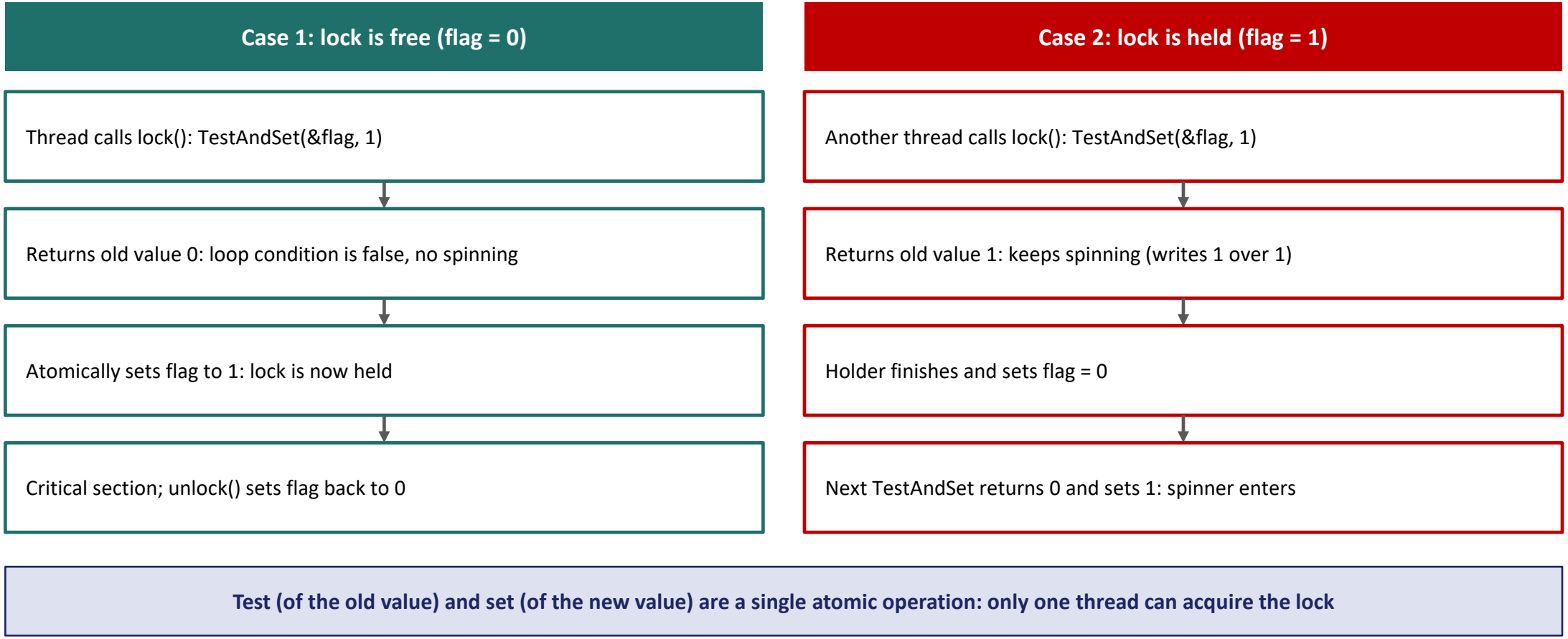
```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0: lock is available, 1: lock is held
    lock->flag = 0;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Why the Spin Lock Works: Two Cases



Evaluating Spin Locks

Correctness: YES

The spin lock allows only a single thread into the critical section at a time.

Fairness: NO

No guarantee that a waiting thread ever gets in. A thread spinning may spin forever under contention: starvation is possible.

Performance: depends

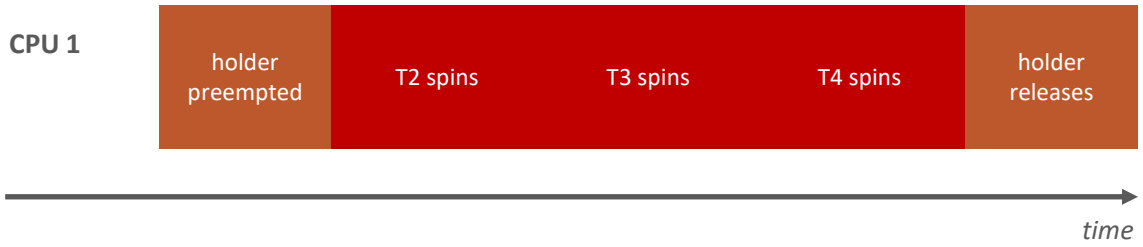
Painful on a single CPU; reasonable on multiple CPUs when threads roughly match CPUs and critical sections are short.

Needs a preemptive scheduler

The timer must interrupt the spinning thread to run another thread. Without preemption a spinner on one CPU never gives the CPU up, so spin locks make little sense.

Spin Lock Performance: One CPU vs Many CPUs

Single CPU: N-1 spinners each burn a whole time slice



Multiple CPUs: the wait is short



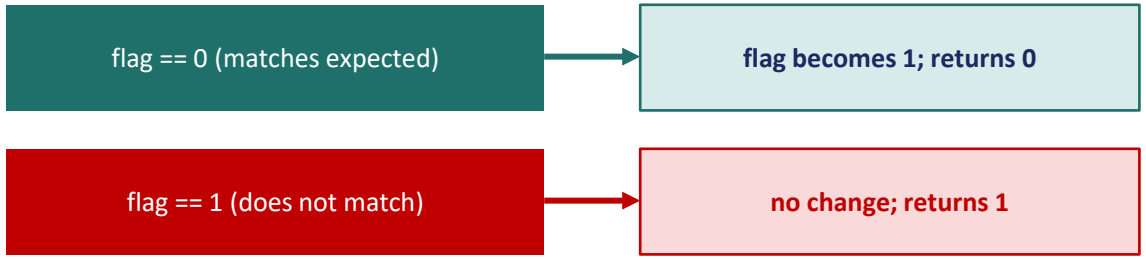
- Single CPU: if the holder is preempted, the scheduler may run all **N - 1** other threads, each spinning for a full time slice
- Multiple CPUs: Thread A on CPU 1 holds the lock; B on CPU 2 spins only briefly if the critical section is **short**
- Spin locks work reasonably well when the number of threads roughly equals the number of CPUs
- Spinning for a short time on another CPU wastes few cycles

Hardware Support: Compare-and-Swap

- Another primitive: **compare-and-swap** (SPARC); **compare-and-exchange** (x86)
- Test whether the value at ptr equals expected
- If so, update ptr to new; if not, do nothing
- Either way, return the original value, so the caller knows if it succeeded

```
int CompareAndSwap(int *ptr, int expected, int new) {
    int original = *ptr;
    if (original == expected)
        *ptr = new;
    return original;
}
```

CompareAndSwap(&flag, 0, 1) when flag is 0 or 1



The whole compare-then-store runs atomically

Building a Lock with Compare-and-Swap

- Replace only the lock() routine of the test-and-set spin lock
- Try to swap flag from **0** to **1**; it works only if the lock is free
- Returns 0: we acquired it. Returns 1: held, spin again
- init and unlock are unchanged
- Behavior is identical to the test-and-set spin lock we analyzed

```
void lock(lock_t *lock) {
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)
        ; // spin
}
```

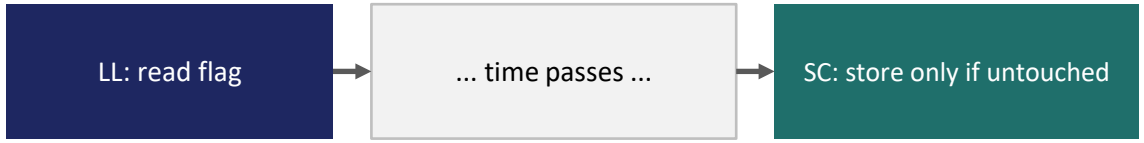
	TestAndSet	CompareAndSwap
Spin condition	TestAndSet(&flag, 1) == 1	CompareAndSwap(&flag, 0, 1) == 1
Writes 1 when	always (writes 1 over 1)	only if flag was 0
Result as a lock	spin lock	same spin lock
Power	basic	more powerful (lock-free code)

Hardware Support: Load-Linked and Store-Conditional

- A pair of instructions that work together (MIPS; also Alpha, PowerPC, ARM)
- Load-linked: an ordinary load that also **remembers** the address
- Store-conditional: succeeds (returns 1, stores the value) only if **no intervening store** to that address happened
- Otherwise it fails: the value is not updated and it returns 0

```
int LoadLinked(int *ptr) {
    return *ptr;
}

int StoreConditional(int *ptr, int value) {
    if (no update to *ptr since LL to this addr) {
        *ptr = value;
        return 1; // success!
    } else {
        return 0; // failed to update
    }
}
```



Building a Lock with LL/SC

- Spin until the flag reads 0 (lock not held)
- Then try to claim it with store-conditional
- Success: flag is atomically 1, return: lock acquired
- Failure: someone else got in between: loop and try again
- Challenge: also try the shorter version using || and !

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // spin until it's zero
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // if set-to-1 was success: done
                    // otherwise: try again
    }
}
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Concise equivalent (suggested by a student)

```
void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) ||
           !StoreConditional(&lock->flag, 1))
        ; // spin
}
```

How a Store-Conditional Fails

- Both threads execute load-linked and see 0
- Only one store-conditional can succeed
- The second finds that flag was updated since its load-linked, so it fails and retries
- Exactly one thread acquires the lock: mutual exclusion

Step	Thread 1	Thread 2	flag
1	LoadLinked: returns 0		0
2	interrupt: switch to T2		0
3		LoadLinked: returns 0	0
4		**StoreConditional: success (1)**	**1**
5	interrupt: switch to T1		1
6	**StoreConditional: FAILS (0)**		**1**
7	loop: LoadLinked returns 1, spins	in critical section	1

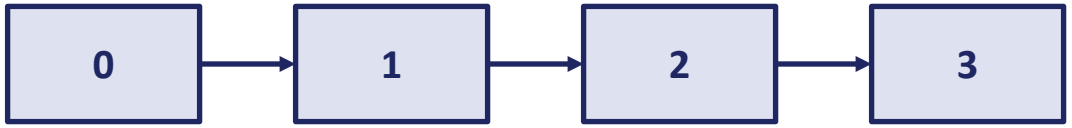
T1's store-conditional fails because T2 wrote flag after T1's load-linked

Hardware Support: Fetch-and-Add

- One final primitive: **fetch-and-add**
- Atomically increments a value and returns the old value at that address
- Each caller gets a different value: the old value of the counter
- This property lets us build a lock that is fair

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

Three threads call FetchAndAdd(&ticket) one after another



value of ticket after 0, 1, 2, 3 calls



Ticket Locks

- Introduced by Mellor-Crummey and Scott
- Two variables: **`ticket`** (next number to hand out) and **`turn`** (whose turn it is)
- lock(): **`myturn = FetchAndAdd(&ticket)`** then spin until **`turn == myturn`**
- unlock(): **`turn = turn + 1`**; the next ticket holder may enter

```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;

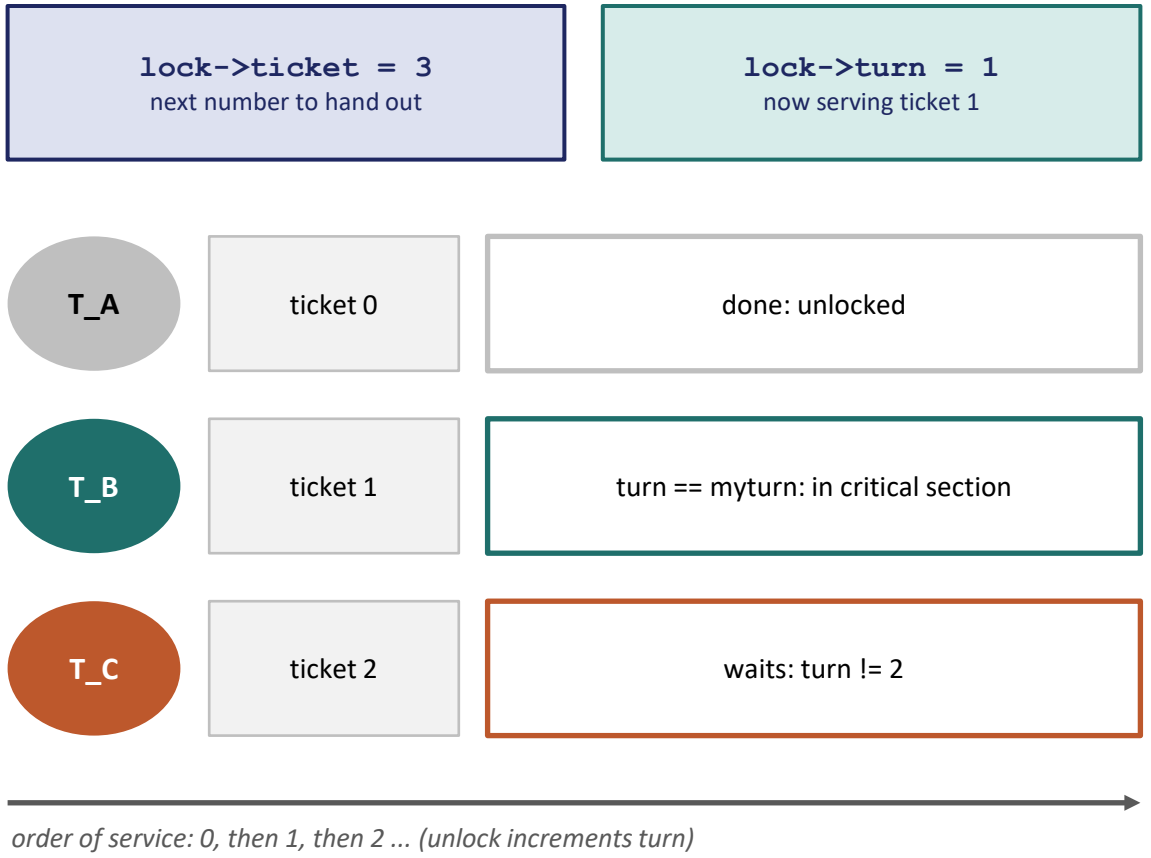
void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn   = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn)
        ; // spin
}

void unlock(lock_t *lock) {
    lock->turn = lock->turn + 1;
}
```

Ticket Lock in Action: Progress for Everyone

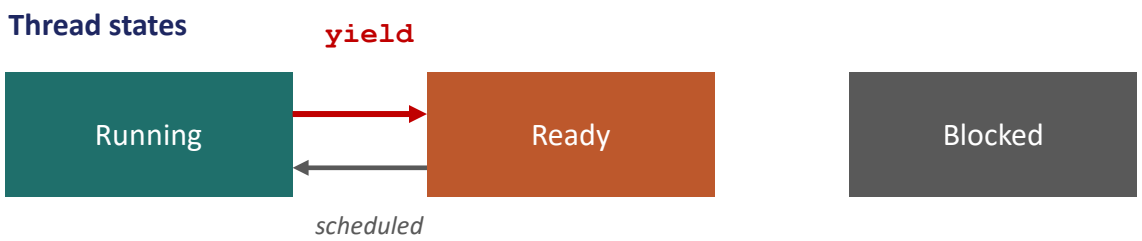
- Each thread gets a ticket and is served in ticket order
- Key difference: **progress is guaranteed** for every thread
- Once a thread has its ticket, it will be scheduled eventually, after those in front have passed through
- Test-and-set spinners, by contrast, could spin forever while others repeatedly acquire and release



Too Much Spinning: Just Yield

- Hardware-only locks are simple and correct, but can be very inefficient
- Thread 0 holds the lock and is interrupted; thread 1 spins for a whole time slice, and so on
- Crux: how can we avoid needlessly spinning on the CPU?
- First try: when about to spin, **give up the CPU** with yield()
- yield() is an OS call that moves the caller from running to ready

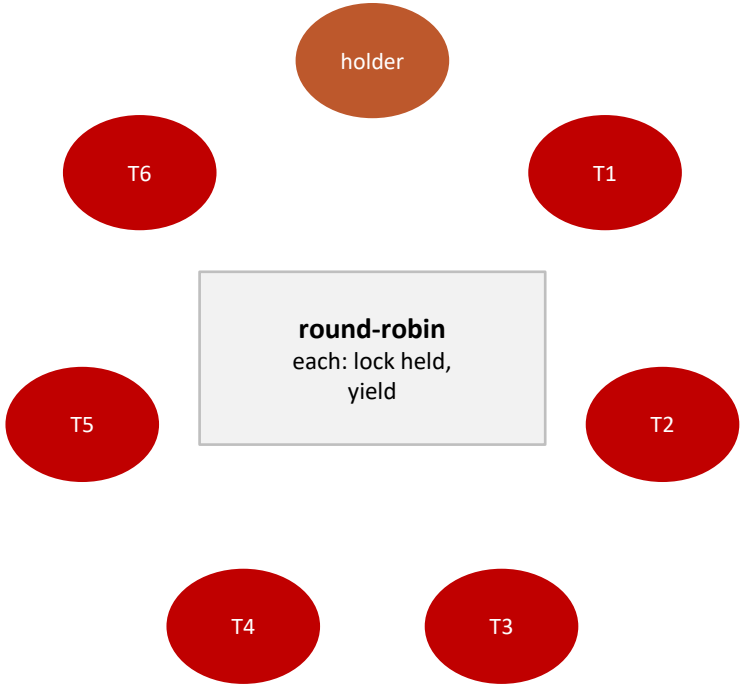
```
void init() {
    flag = 0;
}
void lock() {
    while (TestAndSet(&flag, 1) == 1)
        yield(); // give up the CPU
}
void unlock() {
    flag = 0;
}
```



yield deschedules the caller and lets another thread run

Yielding Is Better, but Still Not Enough

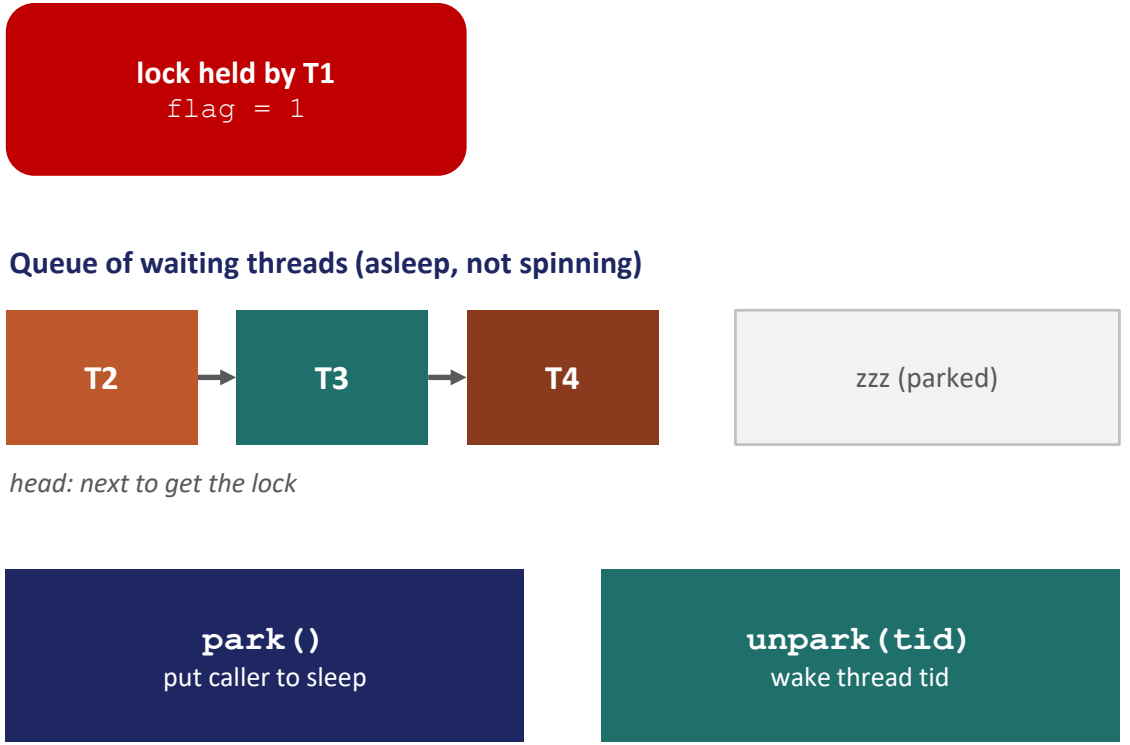
- Two threads on one CPU: works well; the waiter yields and the holder finishes
- 100 threads contending: holder is preempted, the other 99 each run, find the lock held, and yield
- Better than 99 wasted time slices, but each run-and-yield costs a **context switch**
- Worse: **starvation**; a thread may be stuck in an endless yield loop while others enter and exit the critical section
- We need an approach that addresses starvation directly



99 threads each run, test, yield: 99 context switches before the holder runs

Using Queues: Sleeping Instead of Spinning

- Previous locks leave too much to chance: the scheduler decides who runs next
- A bad choice means spinning or immediately yielding: waste, and no starvation prevention
- Explicit control: keep a **queue** of waiting threads and choose who gets the lock next
- Needs OS support. Solaris: park() puts the caller to sleep; unpark(threadID) wakes that thread



Queue Lock: Test-and-Set, Queue, Park and Unpark

```
typedef struct __lock_t {
    int flag; int guard; queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0; m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}
```

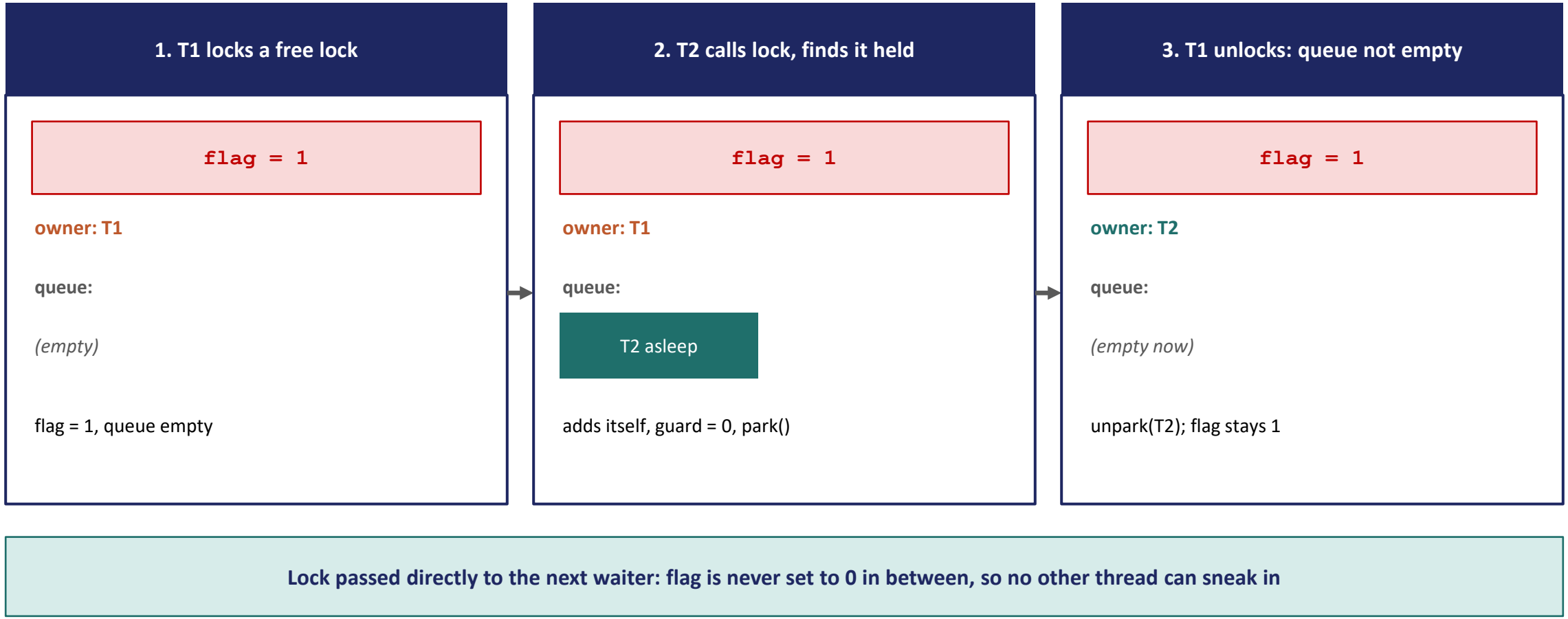
```
void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock
    else
        unpark(queue_remove(m->q));
        // hold lock (for next thread!)
    m->guard = 0;
}
```

guard: a small spin lock around flag and queue operations; spin time is tiny

lock held: add self to the queue, release guard, park()

unlock with waiters: pass the lock directly; flag stays 1

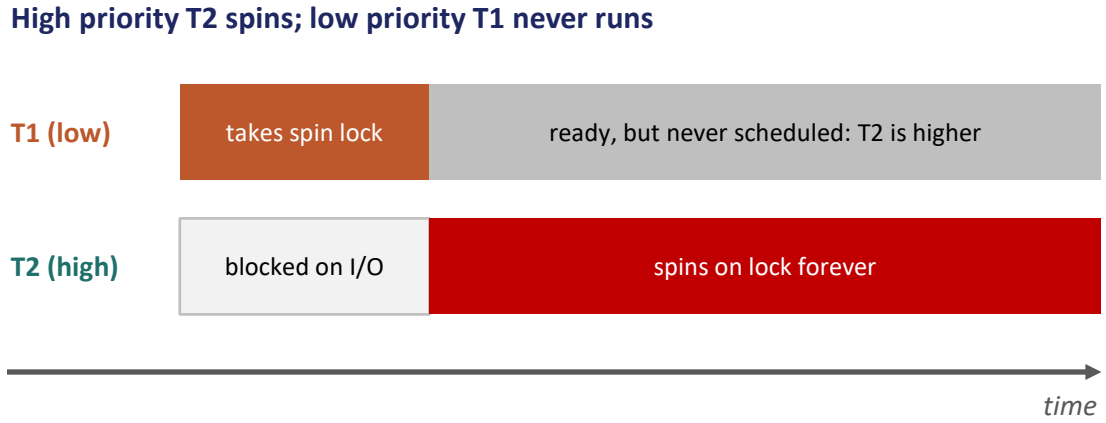
Queue Lock Walk-Through



Lock passed directly to the next waiter: flag is never set to 0 in between, so no other thread can sneak in

Aside: Priority Inversion

- Another reason to avoid spin locks: correctness, not just performance
- Thread 2 has high priority, thread 1 low; the scheduler always prefers T2 when both runnable
- T1 takes a spin lock; T2 wakes up and spins on it; T1 can never run to release it: the system is hung
- Seen on Earth and on Mars (Pathfinder)
- Fixes: avoid spin locks; **priority inheritance** (boost the lock holder); give all threads the same priority



Priority inheritance
While T2 waits for a lock held by T1, T1 temporarily runs at T2's priority: it finishes the critical section, releases the lock, and drops back.

Summary

- Real locks combine hardware support (atomic instructions) with OS support (park/unpark, futex)
- Details differ across systems and are heavily tuned; read the Solaris and Linux sources

Lock	Mutual excl.	Fair	Waits by
flag (load/store)	**No**	-	spin
test-and-set / CAS / LL-SC	Yes	No	spin
ticket (fetch-and-add)	Yes	**Yes**	spin
test-and-set + yield	Yes	No	yield
queue + park/unpark	Yes	**Yes**	**sleep**

progression: each design fixes a weakness of the one before it

Thank You!!